A Blockchain-Based Resource Allocation Mechanism for Edge AI Networks

Your Name

Abstract—The rapid development of large language models has led to the concentration of AI computing power among a few technology giants, while vast idle edge computing resources remain underutilized due to insufficient trust and incentives. This paper proposes BC-RA, a blockchain-based resource allocation mechanism for edge AI networks. Rather than pursuing strict mechanism-theoretic optimality, BC-RA prioritizes deployability, individual rationality (IR), and system composability. The system adopts a tri-layer architecture integrating on-chain smart contracts, off-chain matching engines, and a verification layer. We design a reputation-aware greedy matching algorithm with max-cost-based uniform pricing that balances allocation efficiency with long-term fairness while guaranteeing IR for both requesters and providers. A privacy-preserving task distribution strategy combining off-chain storage, on-chain commitment, and targeted key distribution is proposed, reducing end-to-end latency by approximately 30% for large-scale tasks. We further develop a layered verification framework leveraging EigenLayer, comprising TIQE-based fraud detection with active blacklisting, an original Tensor Hash Chain (THC) baseline for deterministic dispute escalation, and a prefill-focused tolerance-aware TSTC enhancement for heterogeneous execution environments. Experimental results validate the effectiveness of the proposed mechanisms across varying supply-demand conditions.

Index Terms—Blockchain,resource allocation, decentralized AI, privacy preservation

I. Introduction

In recent years, Large Language Models (LLMs), exemplified by ChatGPT, have triggered a revolutionary transformation in the field of artificial intelligence [1]. These models demonstrate powerful natural language understanding and generation capabilities, exhibiting exceptional performance in tasks such as text creation, code writing, and knowledge-based question answering. However, the training and inference of LLMs require substantial computational resources, leading to the gradual concentration of AI computing power in the hands of a few technology giants [2]. This centralization trend has raised widespread concerns: on one hand, ordinary consumers struggle to afford the high costs of cloud-based inference; on the other hand, issues of data privacy and algorithmic transparency have become increasingly prominent [3].

Meanwhile, a vast amount of idle edge computing resources exists globally. Consumer devices such as smartphones, personal computers, and gaming consoles remain idle most of the time, with their potential computing power far from being fully utilized [4]. However, the owners of these devices lack the motivation to contribute their idle resources, while users in need of computing power find it difficult to trust unfamiliar computing nodes. This mutual lack of trust and insufficient incentives between supply and demand constitutes the core obstacle to edge computing resource sharing.

To address these challenges, researchers have explored two directions. Edge AI aims to bring AI capabilities to the network edge, efficiently running AI models on resource-constrained devices through techniques such as model compression and knowledge distillation [5]. Decentralized AI focuses more on the decentralization of control, establishing trustless collaboration mechanisms through cryptographic and consensus algorithms based on blockchain technology, while emphasizing data privacy protection and incentive mechanism design [6]. The open-source project EXO [7] demonstrates the feasibility of collaboratively running LLM inference on heterogeneous device clusters, providing a technical foundation for decentralized AI inference [8].

Blockchain technology offers unique advantages for building decentralized AI systems [6], [9]. Its immutable ledger ensures transparent and trustworthy transaction records, smart contracts can automatically execute predefined incentive rules, and on-chain reputation systems provide quantifiable evaluation criteria for participant behavior [10]. Based on these characteristics, blockchain can effectively address the trust and incentive challenges in edge computing resource sharing [11]. In recent years, research has begun to explore blockchain-based decentralized AI platforms. For instance, PolyLink [12] proposes a decentralized edge AI platform supporting both single-device and cross-device LLM inference, achieving inference quality evaluation through the TIQE protocol that combines a lightweight cross-encoder model with LLM-as-a-Judge [13], and designs a token-based dynamic incentive model. This work provides an important technical foundation for decentralized AI inference.

However, existing solutions still have limitations. Taking PolyLink as an example, although it has made contributions in inference quality evaluation and incentive mechanism design, the following aspects still need improvement: (1) Task matching mechanism: The work does not disclose detailed task matching and distribution mechanisms, and the direct packaging distribution strategy may not adapt to the heterogeneity of task requirements and resource characteristics in real-world scenarios; (2) Pricing mechanism: The work does not introduce a game-theoretic process between major participants, lacking a price discovery mechanism that guarantees individual rationality, which may lead to inefficient resource allocation; (3)

Affiliation/Project Funding (delete this line if not applicable)

Privacy protection: The privacy protection scheme for user tasks and data is not fully elaborated, creating the possibility of platform privacy theft; (4) Accountability localization: Although the TIQE protocol can effectively evaluate task quality, when multiple nodes collaboratively execute tasks, if a node misbehaves, existing solutions struggle to precisely locate the responsible node, failing to achieve fine-grained malicious behavior tracking.

To address these challenges, this paper proposes a Blockchain-based Resource Allocation mechanism (BC-RA). The system adopts a tri-layer architecture integrating on-chain smart contracts, off-chain matching engines, and a dedicated verification layer. The main contributions include:

- Designing a reputation-aware greedy matching algorithm with max-cost-based uniform pricing: The matching engine prioritizes providers based on a composite reputation score derived from stake, historical service records, and fraud counts, while the pricing mechanism guarantees individual rationality for both requesters and providers. Experimental results demonstrate that social welfare increases with participant scale, resource utilization approaches saturation under balanced supply-demand conditions, and the pricing weight parameter effectively shifts surplus allocation between the two sides.
- Proposing a privacy-preserving task distribution strategy based on off-chain storage, on-chain commitment, and targeted key distribution: Task data is encrypted with symmetric keys and uploaded to IPFS [14], with only hash commitments recorded on-chain. Decryption keys are distributed to winning providers via asymmetric encryption. Experimental results show that this approach reduces end-to-end latency by approximately 30% and decreases upload bandwidth overhead to $1/|M_j|$ compared to direct transmission schemes for large-scale tasks.
- Developing a layered verification framework leveraging EigenLayer [15]: The framework comprises three progressive schemes: TIQE-based fraud detection with active blacklisting for routine spot-checking, a chain-based dispute-escalation verifier combining the original Tensor Hash Chain (THC) baseline with the tolerance-aware Tolerance-Aware Sampled Tensor Chain (TSTC) enhancement, and zero-knowledge verification via zkLLM [16] and DSperse [17] as a long-term evolution direction.

Our design prioritizes engineering feasibility and deployability over strict mechanism-theoretic optimality. We do not claim truthfulness or welfare maximization; instead, BC-RA provides a practical, composable framework with explicit design trade-offs documented throughout.

II. Related Work

Blockchain-Based Resource Allocation and Matching Mechanisms. Blockchain technology provides trusted infrastructure for task matching and resource allocation in decentralized environments [11]. Li et al. [18] propose a double auction mechanism for edge computing resource allocation, achieving automated resource trading through smart contracts. Habiba et al. [19] design a dynamic resource allocation model based on repeated auctions, considering load-aware pricing strategies. While these auction-based solutions offer theoretical optimality guarantees (e.g., strategy-proofness, welfare maximization), they often entail significant system complexity, on-chain computational costs, and implementation challenges in heterogeneous edge environments. In contrast, BC-RA adopts a simpler greedy matching approach that sacrifices strict auction optimality in favor of engineering simplicity, low on-chain settlement costs, and straightforward integration with reputation systems. BC-RA designs a reputation-aware greedy matching algorithm that prioritizes providers based on a composite score derived from stake, historical service records, and fraud counts, combined with a max-cost-based uniform pricing mechanism that guarantees individual rationality for both sides.

Privacy-Preserving Mechanisms in Decentralized Computing. Protecting the privacy of user tasks and data is a core challenge in decentralized computing environments. Javet et al. [20] propose the Edgelet computing paradigm, combining Trusted Execution Environments (TEE) to achieve privacy-preserving distributed data processing. Homomorphic encryption allows computation directly on ciphertexts without decryption [21]. However, TEE solutions depend on specific hardware such as Intel SGX and ARM TrustZone, resulting in poor cross-device compatibility [22]; while homomorphic encryption provides strong privacy guarantees, its computational overhead is extremely high, with significant performance bottlenecks in practical applications. BC-RA adopts a lightweight privacy-preserving strategy combining off-chain storage via IPFS [14], on-chain hash commitments, and targeted key distribution via asymmetric encryption, achieving dual-layer access control at both the storage and key levels without requiring specialized hardware.

Verifiable Computation and Fraud Detection. Verifiable computation aims to ensure the correctness of computation results and prevent malicious nodes from providing incorrect results. Sun et al. [16] first achieve zero-knowledge proofs for large language model inference, proving the correctness of the inference process without revealing model parameters or input data. Ivanov et al. [17] propose the DSperse framework, reducing computational overhead through sharded verification [23], [24]. However, while ZKML provides cryptographic-level security, the computational overhead of proof generation is extremely high; for large-scale model inference, proof time may far exceed the inference itself, making it impractical in real-world applications. Additionally, when multiple nodes collaboratively execute tasks, existing solutions struggle to precisely locate malicious nodes while remaining robust to honest numerical drift across heterogeneous hardware. BC-RA addresses this through a layered verification strategy: TIQE-based fraud detection [13] with active black-

listing for routine spot-checking, and dispute-escalation chain verification using the original THC baseline with a tolerance-aware TSTC enhancement.

EigenLayer and Security Service Outsourcing. EigenLayer [15] is an Ethereum-based restaking protocol that allows Ethereum validators to provide security guarantees for other services requiring economic security, called Actively Validated Services [25], [26]. EigenLayer's core innovation lies in achieving capital reuse, enabling validators to protect multiple AVSs with the same capital while maintaining Ethereum staking, thereby reducing the startup costs for new services. BC-RA leverages EigenLayer as the infrastructure for outsourcing verification services, where validators register as AVS operators and their staked assets provide economic guarantees for verification correctness, enabling the system to obtain scalable security services without building its own validator network.

Compared with existing solutions, BC-RA addresses the gaps in reputation-aware matching, privacy protection during task distribution, and precise accountability localization through its tri-layer architecture integrating on-chain contracts, off-chain matching engines, and a dedicated verification layer.

III. System Overview

This chapter presents a systematic description of the overall architecture and operational workflow of the proposed Blockchain-based Resource Allocation (BC-RA) mechanism. Unlike traditional centralized platforms, BC-RA adopts a collaborative on-chain and off-chain design to decouple key stages including task publication, resource matching, privacy-preserving execution, and result verification and settlement. Such a design enables BC-RA to achieve efficiency, fairness, and verifiability simultaneously in a decentralized Edge AI environment. This chapter first introduces the system participants and their roles, then details the overall architecture, and finally explains the complete operational workflow.

A. System Participants and Roles

The BC-RA system consists of three types of participants, each assuming distinct yet mutually constraining roles.

Task Requesters: They have demands for AI inference or computation tasks and are willing to pay for computational services. Their primary concerns include task completion latency, pricing rationality, and the privacy protection of task data in a decentralized environment.

Resource Providers: They own heterogeneous edge computing resources, such as personal computers, mobile devices, or edge servers. By registering their resource capabilities and participating in bidding, they provide computational services in exchange for economic rewards. The long-term returns of resource providers is closely related to their historical execution behavior, reputation level, and honesty in task completion.

Third-Party Entities: They provide the trusted infrastructure supporting the system's operation. These entities include blockchain smart contracts, the off-chain matching engine, and validators. Smart contracts maintain task and resource states, escrow funds, and automatically enforce settlement and penalty rules. The matching engine performs efficient task–resource matching off-chain, while validators verify execution results and arbitrate disputes when necessary, thereby enabling accountability in a decentralized setting.

These three types of participants are functionally independent and constrained through blockchain and cryptographic mechanisms, allowing the system to achieve effective incentives, fairness, and scalability without requiring full trust in any single entity.

B. Overall System Architecture

From an architectural perspective, BC-RA adopts a layered design that integrates an on-chain layer, an off-chain layer, and a verification layer. The core idea is to ensure transparency and trustworthiness of critical states while offloading computation-intensive logic from the blockchain, thus balancing security and system efficiency.

The on-chain Layer: It is composed of a set of functionally specialized and cooperative smart contracts, forming the foundation of system trust and incentive enforcement. The TaskManager.sol contract maintains task registration states and basic task descriptions, ensuring the traceability of task declarations. The ResourceManager.sol contract records resource providers' device capabilities, identity bindings, and staking-related information. The OrderManager.sol contract manages buy bids from task requesters and sell bids from resource providers, and accepts matching results submitted by the off-chain matching engine, enabling atomic updates of order states. In addition, settlement and penalty logic is automatically executed by corresponding contracts, guaranteeing transparency and immutability in fund distribution and reputation updates.

The Off-chain Layer: It is centered around the matching engine. By monitoring events emitted by on-chain contracts, the engine aggregates unmatched tasks and available resources in real time and performs complex sorting, filtering, and combination operations off-chain, thereby avoiding the high computational cost of on-chain execution. The matching engine only produces final matching results and does not participate in fund custody or state adjudication; its outputs must be confirmed and recorded by on-chain contracts.

The Verification Layer: It consists of the validators and the arbitration contract Arbitration.sol. It is responsible for spot-checking or verifying the results of task execution after completion. When anomalies or disputes are detected, validators re-examine the execution process and submit verification results on-chain. Based on these results, the arbitration contract triggers the corresponding penalty or compensation mechanisms, enabling accountability and dispute resolution in a decentralized environment.

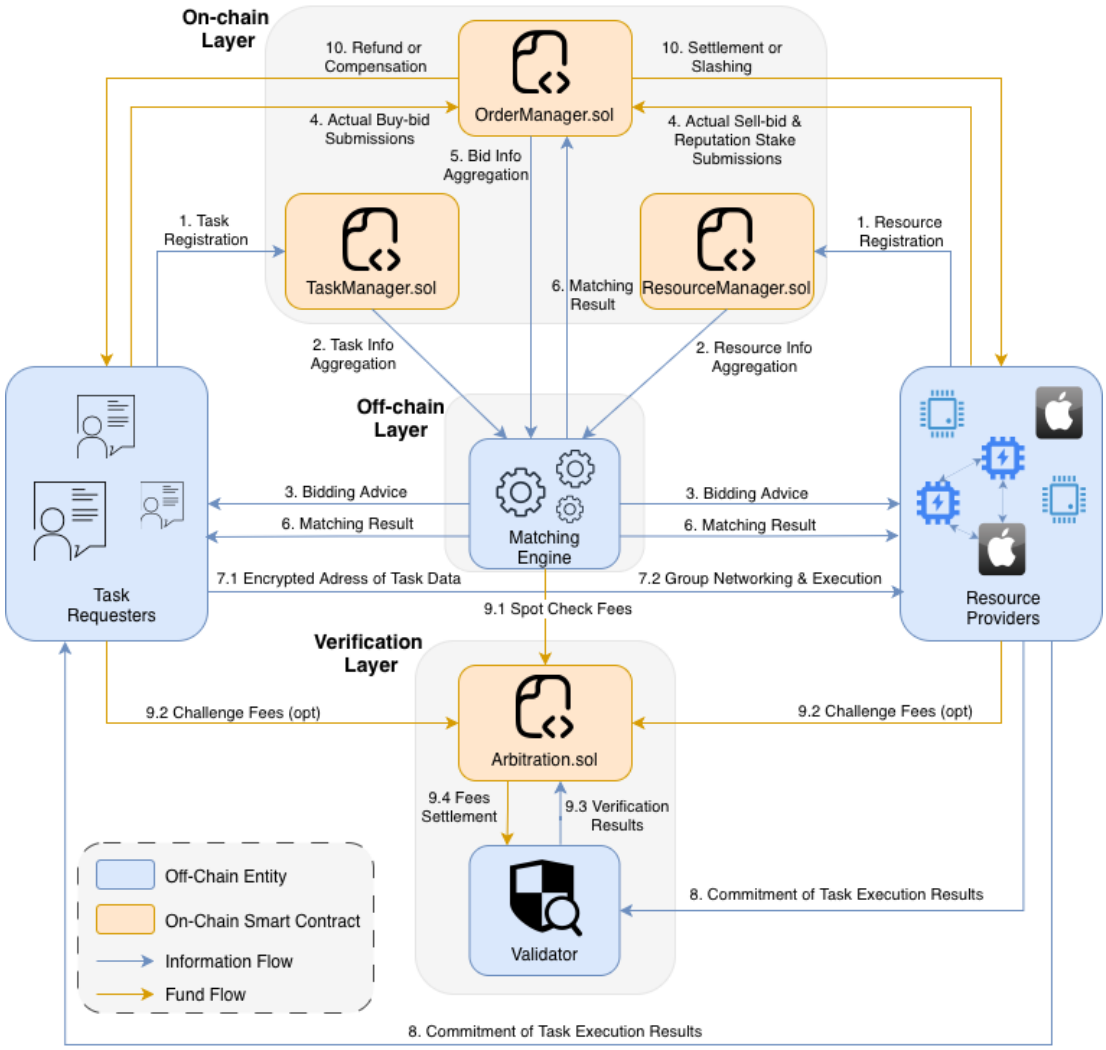

Fig. 1. BC-RA System Architecture and Workflow Diagram. Blue boxes represent off-chain entities, orange boxes represent on-chain smart contracts, blue arrows indicate information flow, and orange arrows indicate fund flow. TaskManager.sol and ResourceManager.sol receive task and resource registration information from requesters and providers, respectively, and broadcast the data to the off-chain matching engine. OrderManager.sol maintains the order book for tasks matching and funds settlement execution. Arbitration.sol handles dispute resolution and manages all fees related to verification.

C. System Workflow Overview

BC-RA operates by following the complete lifecycle of a user's computational task, from deployment to final settlement, and includes several logically interconnected phases. We break this process down into 10 steps in Figure 1.

1) Task and Resource Registration (Step 1): The task requesters register basic task information through the TaskManager contract, declaring the existence of their tasks and their specific requirements. The resource providers register their computing power, identity binding, and staking status through the ResourceManager contract. At this stage, both parties only disclose the necessary abstract information to support subsequent bidding and matching.
2) Task and Resource Registration (Step 1):The task requesters register basic task information through the TaskManager contract, declaring the existence of their tasks and their specific requirements. The resource providers register their computing power, identity binding, and staking status through the ResourceManager contract. At this stage, both parties only disclose the necessary abstract information to support subsequent bidding and matching.
3) Information Aggregation (Step 2): The on-chain smart contracts broadcast information on registered tasks and resources, and the off-chain matching engine aggregates related information through events, providing complete input for its subsequent matching calculations.
4) Bidding Advice and Bid Submission (Steps 3–4): Based on the aggregated market information, the matching engine provides bidding advice to both task requesters and resource providers (Step 3). After receiving the advice, task requesters submit their actual buy-bid orders, and resource providers submit

their sell-bid orders along with reputation stakes to the OrderManager contract (Step 4), forming a market order book.

5) Bid Aggregation and Off-chain Matching (Steps 5–6): The OrderManager contract aggregates all submitted bids and broadcasts the information to the matching engine (Step 5). The matching engine then calculates the matching results off-chain and submits them to the OrderManager contract in a deterministic allocation format, whereby it is recorded on-chain for confirmation and distribution (Step 6).
6) Privacy-Preserving Task Execution (Step 7): Once a match is found, the execution proceeds in two phases: First, the task requester distributes the encrypted address of task data to the selected resource providers (Step 7.1). Then, the winning providers establish a temporary compute cluster, collaboratively execute the task (Step 7.2). Unsuccessful nodes and off-chain components have no access to the plaintext task data.
7) Result Submission, Verification, and Settlement (Steps 8–10): Once resource providers have completed the computational task, they submit a commitment to the execution result on-chain (Step 8). The verification phase involves: system-initiated spot checks with prepaid fees (Step 9.1); optional challenge fees paid by either party to trigger additional verification (Step 9.2); validators submitting verification conclusions (Step 9.3); and the arbitration contract settling all related fees (Step 9.4). Based on the verification results, the system executes final settlement: task requesters receive refunds or compensation if fraud is detected (Step 10), while resource providers receive payments or face stake slashing depending on the outcome. These status changes will directly affect participants' economic gains and future matching opportunities.

D. Design Objectives and Core Challenges

The above system architecture and workflow are based on the following design goals:

- Balancing efficiency and long-term fairness: The system ensures that high-value tasks are completed efficiently and prioritized, while preventing computing resources from being concentrated in a few nodes for extended periods, providing sustainable participation opportunities for all nodes.
- Privacy protection for participants: Task data and intermediate results are guaranteed to be accessible only to authorized resource providers, avoiding information leakage due to blockchain transparency or third-party intervention.
- Verifiability and accountability in a decentralized environment: The system can effectively verify execution results and accurately identify anomalous nodes without relying on centralized oversight.

These goals directly determine the design of the system's encryption mechanism, matching and pricing strategies, and verification scheme. We will elaborate on the core mechanism design of BC-RA in the next chapter.

IV. Core Mechanism Design

This chapter presents the core mechanisms that enable resource allocation and task execution in our BC-RA system. Given the decentralized setting and practical engineering constraints, our focus is not on proving strict game-theoretic optimality, but on making explicit design choices that balance implementability, scalability, and security. Following the system architecture and workflow in Chapter 3, we first describe the reputation-aware off-chain matching mechanism and then the max-cost-based uniform pricing mechanism.

TABLE I
Symbols Used in the Matching and Pricing Mechanism

Symbol	Explanation
S, B	Provider set; task set (one matching round)
n, m	Number of providers/tasks
a_i	Provider i unit cost (or ask lower bound)
c_i	Provider i available capacity
r_i	Provider i reputation score, $r_i \in [0,1]$
s_i	Provider i stake (escrowed on-chain)
t_i	Provider i historical completed-task count
f_i	Provider i confirmed fraud count (by arbitration)
$\tilde{s}_i$	Normalized stake score, $\tilde{s}_i \in [0,1]$
$\tilde{t}_i$	Normalized completion score, $\tilde{t}_i \in [0,1]$
$\tilde{f}_i$	Normalized fraud score, $\tilde{f}_i \in [0,1]$
s_{ref}	Reference stake in log-normalization, $s_{\text{ref}} > 0$
τ	Saturation parameter in $\tilde{t}_i$, $\tau > 0$
κ	Penalty parameter in $\tilde{f}_i$, $\kappa > 0$
w_s, w_t, w_f	Reputation weights, $w_s + w_t + w_f = 1$
F_{ban}	Optional fraud threshold for blacklisting
d_j	Task j demand
b_j	Task j total budget
θ_j	Task j unit valuation (b_j/d_j)
x_{ij}	Allocation from provider i to task j
M_j	Winner set for task j ($x_{ij} > 0$)
p_j	Uniform unit payment price for task j
α	Pricing weight, $\alpha \in [0,1]$
$\bar{a}^{\text{ub}}$	Robust upper bound for asks in pricing guard
$\tilde{a}_j^{\max}$	Guarded max ask for task j, $\min(\max_{i \in M_j} a_i, \bar{a}^{\text{ub}})$
d_j^{rep}	Reported demand of task j
$\bar{d}^{\text{ub}}$	Robust upper bound for demand reports
$\tilde{d}_j$	Guarded demand for matching/settlement, $\min(d_j^{\text{rep}}, \bar{d}^{\text{ub}})$
$X = [x_{ij}]$	Allocation matrix

A. Reputation-Aware Off-Chain Matching

The matching mechanism corresponds to steps 2-6 in Figure 1. Our goal is to compute deterministic and reproducible allocation results at low on-chain costs while improving execution reliability by prioritizing reputable providers. To this end, BC-RA offloads computationally intensive sorting and allocation tasks to an off-chain matching engine, while maintaining operations such as order state transitions, escrow, and result commitments on-chain (e.g., via OrderManager.sol). The algorithm 1 summarizes the matching process, and the key symbols are listed in Table I.

We consider a match round with a provider set $S = \{1, \ldots, n\}$ and a task set $B = \{1, \ldots, m\}$. Each provider $i \in S$ registers its unit cost a_i, available capacity c_i, and

Algorithm 1 Reputation-Prioritized Greedy Matching

Require: Providers $S = \{(a_i, c_i, r_i)\}_{i=1}^{n}$; Tasks $B = \{(d_j, \theta_j)\}_{j=1}^{m}$
Ensure: Allocation $X = [x_{ij}]$; winner sets $\{M_j\}_{j=1}^{m}$

Feasibility check: compute $\sum_{i=1}^{n} c_i$ and $\sum_{j=1}^{m} d_j$ for logging; continue with task-wise allocation
Sort providers by r_i descending, then a_i ascending
Sort tasks by θ_j descending, then d_j ascending
Initialize $c_i^{\text{rem}} \leftarrow c_i$, $d_j^{\text{rem}} \leftarrow d_j$, $x_{ij} \leftarrow 0$, $M_j \leftarrow \emptyset$
for each task j in sorted tasks **do**
  for each provider i in sorted providers **do**
    if $d_j^{\text{rem}} \le 0$ **then**
      break
    end if
    if $c_i^{\text{rem}} \le 0$ **then**
      continue
    end if
    if $\theta_j > a_i$ **then**
      $\delta \leftarrow \min(c_i^{\text{rem}}, d_j^{\text{rem}})$
      if $\delta > 0$ **then**
        $x_{ij} \leftarrow \delta$; $c_i^{\text{rem}} \leftarrow c_i^{\text{rem}} - \delta$; $d_j^{\text{rem}} \leftarrow d_j^{\text{rem}} - \delta$
        $M_j \leftarrow M_j \cup \{i\}$
      end if
    end if
  end for
  if $d_j^{\text{rem}} > 0$ **then**
    mark task j as failed
    continue
  end if
end for
return $X, \{M_j\}$

reputation score $r_i \in [0, 1]$. Each task $j \in B$ specifies a demand d_j and a total budget b_j, which induces the unit valuation $\theta_j = b_j/d_j$. The matching engine takes $\{(a_i, c_i, r_i)\}$ and $\{(d_j, \theta_j)\}$ as inputs and outputs an allocation matrix $X = [x_{ij}]$, where $x_{ij} \ge 0$ denotes the amount of computing power allocated from the provider i to task j. The winner set for task j is $M_j = \{i \mid x_{ij} > 0\}$.

In engineering practice, r_i must be computable, updateable, and grounded in on-chain state. BC-RA adopts a three-factor reputation model based on stake–service–penalty. Specifically, provider i posts a stake s_i during registration and bidding; its completed-task count t_i and confirmed fraud count f_i are accumulated by chain contracts (e.g., settlement updates t_i, while arbitration confirms misbehavior, increases f_i, and triggers slashing). To avoid extreme values dominating the ranking, the engine maps (s_i, t_i, f_i) to comparable scores in $[0, 1]$:

$$\tilde{s}_i = \min\left(1, \frac{\log(1 + s_i)}{\log(1 + s_{\text{ref}})}\right), \tag{1a}$$

$$\tilde{t}_i = 1 - \exp\left(-\frac{t_i}{\tau}\right), \tag{1b}$$

$$\tilde{f}_i = \exp\left(-\frac{f_i}{\kappa}\right). \tag{1c}$$

and aggregates them as

$$r_i = \text{clip}\left(w_s \tilde{s}_i + w_t \tilde{t}_i + w_f \tilde{f}_i,\ 0,\ 1\right), \tag{2}$$

where $w_s + w_t + w_f = 1$. Optionally, a hard rule can be enabled: if $f_i \ge F_{\text{ban}}$, the provider is blacklisted (e.g., forced to $r_i = 0$) and restricted from future matching rounds, together with punitive stake slashing. This closes the loop between reputation ranking and on-chain accountability.

The matching engine employs a deterministic greedy strategy to reduce implementation complexity and achieve high throughput. It first records global supply-demand statistics $\sum_{i=1}^{n} c_i$ and $\sum_{j=1}^{m} d_j$, but does not hard-abort the round when supply is insufficient. Providers are sorted by $(r_i \downarrow, a_i \uparrow)$, and tasks are sorted by $(\theta_j \downarrow, d_j \uparrow)$. The engine then iterates over tasks in descending θ_j, and for each task, allocates capacity from providers in sorted order until the demand is met. To avoid economically meaningless allocations, it only assigns from provider i to task j when $\theta_j > a_i$.

Importantly, BC-RA adopts a task-wise feasibility model rather than requiring round-wise feasibility. If a task cannot be fully satisfied after exhausting all eligible providers (i.e., $d_j^{\text{rem}} > 0$), the task is marked as failed and the algorithm continues to the next task. This design permits partial task satisfaction within a matching round, allowing the system to maximize overall resource utilization even when some tasks are infeasible due to insufficient supply or valuation–cost mismatches. The resulting success ratio may be less than 1, which is reflected in our experimental results.

The resulting time complexity is $O(nm)$ in the worst case, which is suitable for event-driven, frequent execution off-chain. Since sorting and traversal are deterministic, the allocation is reproducible and can be committed on-chain for traceability and downstream settlement.

At the on-chain/off-chain boundary, the matching engine listens to aggregation events from TaskManager.sol and ResourceManager.sol, computes $\{(j, M_j, X_{\cdot j})\}$, and submits the structured result to OrderManager.sol at Step 6. The matching engine does not custody funds and does not adjudicate disputes; its output only serves as input to on-chain contracts for settlement, as well as to subsequent privacy-preserving execution and audit procedures.

B. Max-Cost-Based Uniform Pricing

When a task is served by multiple providers, BC-RA adopts a uniform unit price p_j for each task j to simplify on-chain settlement and off-chain reconciliation. Intuitively, p_j must cover the marginal cost among winners and leave some surplus to incentivize participation. We use the following max-cost-based weighted pricing rule:

$$p_j = \alpha \cdot \max_{i \in M_j} a_i + (1 - \alpha) \cdot \theta_j, \tag{3}$$

where $\alpha \in [0, 1]$ is a configurable pricing weight. Algorithm 2 provides the reference procedure.

Algorithm 2 Uniform Pricing with Individual Rationality

Require: Winner sets $\{M_j\}_{j=1}^{m}$; provider costs $\{a_i\}_{i=1}^{n}$; task valuations $\{\theta_j\}_{j=1}^{m}$; weight $\alpha \in [0,1]$
Ensure: Task prices $p = [p_j]$

```
for each task j do
    if M_j = ∅ then
        p_j ← 0
    else
        a_j^max ← max_{i∈M_j} a_i
        p_j ← α · a_j^max + (1 − α) · θ_j
        p_j ← max(a_j^max, min(θ_j, p_j)) {ensure IR}
    end if
end for
return p
```

This design yields two direct engineering benefits. First, a uniform p_j allows OrderManager.sol to settle the task payment atomically as $p_j \cdot d_j$, and to distribute provider revenues linearly by the realized allocations x_{ij}, avoiding per-pair pricing that would inflate on-chain storage and computation. Second, with $p_j \in [\max_{i\in M_j} a_i, \theta_j]$, for any winner $i \in M_j$ we have $(p_j - a_i)x_{ij} \geq 0$, and for the requester we have $(\theta_j - p_j)d_j \geq 0$, ensuring IR on both sides as a minimal economic stability condition.

The parameter α controls how the surplus is split between the two sides. A larger α moves the price closer to the max cost, increasing requester surplus and reducing provider surplus; a smaller α moves the price closer to the task valuation, strengthening provider-side incentives, which can be useful under scarce supply or when attracting highly reputable providers. Since our focus is engineering feasibility rather than equilibrium computation, α is treated as a tunable system parameter (e.g., set by governance or an administrator according to supply–demand conditions).

To move robustness from "experiment patching" into mechanism design, BC-RA adds a guarded reference layer above Eq. (3). For provider ask reports, let $A = \{a_i\}_{i\in S}$. We compute a robust upper bound $\bar{a}^{\mathrm{ub}}$ by one of two governance-selectable rules:

$$\bar{a}^{\mathrm{ub}} = Q_{0.75}(A) + \eta_a(Q_{0.75}(A) - Q_{0.25}(A)), \quad \text{(IQR mode)} \tag{4}$$

$$\bar{a}^{\mathrm{ub}} = Q_{q_a}(A), \quad q_a \in [0.90, 0.99], \quad \text{(percentile mode)} \tag{5}$$

where $Q_q(\cdot)$ is the empirical q-quantile. For each matched task j, the max-cost reference is replaced by

$$\tilde{a}_j^{\max} = \min\left(\max_{i\in M_j} a_i,\ \bar{a}^{\mathrm{ub}}\right), \tag{6}$$

and pricing becomes

$$p_j = \mathrm{clip}\big(\alpha\tilde{a}_j^{\max} + (1-\alpha)\theta_j,\ [\tilde{a}_j^{\max}, \theta_j]\big), \tag{7}$$

which preserves IR while reducing the leverage of extreme ask inflation on uniform prices.

For requester demand inflation, BC-RA applies a demand sanity cap before matching. Let $D = \{d_j^{\mathrm{rep}}\}_{j\in B}$ denote reported demands in a round. We compute $\bar{d}^{\mathrm{ub}}$ using the same robust statistics family:

$$\bar{d}^{\mathrm{ub}} = Q_{0.75}(D) + \eta_d(Q_{0.75}(D) - Q_{0.25}(D)), \quad \text{(IQR mode)} \tag{8}$$

$$\bar{d}^{\mathrm{ub}} = Q_{q_d}(D), \quad q_d \in [0.90, 0.99], \quad \text{(percentile mode)} \tag{9}$$

and enforce the executable rule

$$\tilde{d}_j = \min\big(d_j^{\mathrm{rep}}, \bar{d}^{\mathrm{ub}}\big). \tag{10}$$

The matching engine allocates on $\tilde{d}_j$ (instead of raw d_j^{rep}), and settlement uses the same guarded quantity, so inflated reports cannot scale linearly into allocation pressure.

Deployment-wise, BC-RA exposes two selectable robustness profiles. Profile-A (conservative) uses IQR mode for both Eq. (7) and Eq. (10), giving stronger suppression on tail reports. Profile-B (adaptive) uses percentile mode, giving smoother behavior under demand/price distribution shifts. A stronger optional extension is to combine Eq. (10) with escrow-backed budget consistency (guarded demand additionally bounded by prepaid budget), which provides tighter anti-inflation guarantees but requires extra contract-state checks and thus higher on-chain complexity.

These guards do not claim full strategy-proofness or welfare optimality; instead, they keep the mechanism lightweight (single-pass statistics + deterministic clipping) while introducing explicit, executable anti-manipulation rules at the mechanism layer.

C. Privacy Preservation and Networking Execution

As illustrated in Fig. 2, once matching is finalized, the system enters the privacy-preserving data distribution and task execution phase. Since BC-RA operates in a decentralized environment where on-chain data is visible to all nodes, while task content often involves user privacy or commercially sensitive information, it is necessary to ensure that task plaintext is accessible only to winning providers without compromising the transparency of matching and settlement. This section addresses three sub-problems: task storage, address distribution, and collaborative execution.

1) Privacy-Preserving Task Data Distribution: To reduce on-chain storage overhead and leverage the decentralized storage ecosystem, BC-RA adopts a privacy-preserving strategy of "off-chain storage, on-chain commitment, and targeted distribution." Let m_j denote the plaintext data of task j. During task submission, the requester first generates a symmetric key k_j locally and encrypts m_j:

$$C_j = \mathcal{E}_{\mathrm{sym}}(k_j, m_j), \tag{11}$$

where $\mathcal{E}_{\mathrm{sym}}$ denotes a symmetric encryption algorithm (e.g., AES-256). The requester then uploads the ciphertext C_j to a decentralized storage network(DSN) such as IPFS [14] and obtains a storage locator ℓ_j (i.e., the CID). Only the hash value $H(m_j)$ of the plaintext is recorded on-chain as an existence commitment, rather than storing the

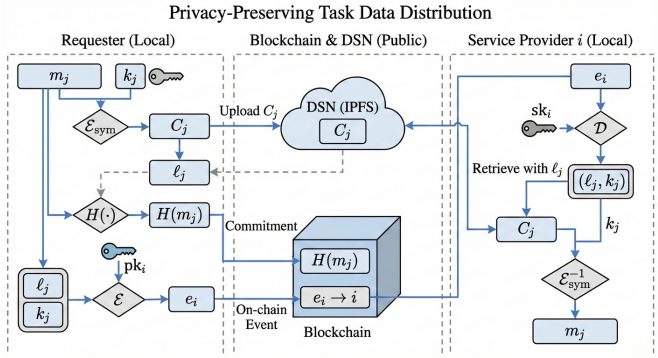

Fig. 2. Privacy-preserving task distribution and collaborative execution workflow.

ciphertext itself, thereby significantly reducing on-chain storage costs.

After matching results are confirmed, the requester retrieves the pre-registered public key pk_i of each winning provider $i \in M_j$ and encrypts the combination of the storage locator and decryption key using asymmetric encryption:

$$e_i = \mathcal{E}(\mathsf{pk}_i, (\ell_j, k_j)), \tag{12}$$

where $\mathcal{E}$ denotes an asymmetric encryption function. The encrypted payload e_i can be delivered to the corresponding provider via on-chain events. Provider i decrypts using its private key sk_i to obtain the storage locator and decryption key:

$$(\ell_j, k_j) = \mathcal{D}(\mathsf{sk}_i, e_i), \tag{13}$$

where $\mathcal{D}$ denotes the corresponding decryption function. The provider then retrieves the ciphertext C_j from the IPFS network and decrypts it locally using k_j to obtain the task plaintext. Since IPFS employs content addressing [14], nodes without ℓ_j cannot locate the data; moreover, even if other nodes obtain the ciphertext, they cannot decrypt it without k_j, thereby achieving dual-layer access control at both the storage and key levels.

To evaluate the cost profile of this design, our experiments compare it against a baseline protocol called Replicated Payload Delivery (RPD), where the requester independently transmits a full encrypted payload to each winner without shared ciphertext publication and targeted key release. The BC-RA design in this section is referred to as Publish-Once, Key-to-Winners (POKW) in the experiments.

2) Networking and Collaborative Execution: After obtaining the task plaintext, winning providers must establish a temporary compute cluster based on network topology to collaboratively execute inference tasks. BC-RA provides different networking strategies for two typical scenarios: local-area networks (LAN) and wide-area networks (WAN).

In LAN environments, providers and requesters reside within the same subnet and can establish high-throughput connections using application-layer protocols such as gRPC or HTTP/2 [27], typically achieving bandwidth of several hundred MB/s, sufficient for tensor transmission in large-scale inference tasks. In WAN environments, cross-network providers employ P2P overlay networks (e.g., libp2p, WebRTC) combined with NAT traversal techniques (STUN, TURN, or ICE) to establish direct connections [28]; if direct connection fails, the system falls back to relay servers. Although WAN bandwidth is typically lower than LAN, optimized data channels can still achieve acceptable throughput levels for transmitting inference results and intermediate tensors. Regardless of the networking approach, communication links should enable TLS or QUIC encryption to ensure confidentiality and integrity during transmission.

Once the compute cluster is established, the winning providers collaboratively execute the inference task according to a predetermined model sharding scheme. Upon completion, each provider returns the inference result to the requester through a secure channel and submits the hash of the final result on-chain for subsequent verification and audit procedures.

D. Verification and Settlement

After task execution is completed, the system must verify execution results to prevent misbehavior by resource providers in the decentralized environment. As illustrated in Fig. 1, the verification layer consists of validators and the arbitration contract Arbitration.sol, responsible for result commitment, spot-check verification, and dispute resolution in Steps 8–10. This section addresses verification triggering and fee management, security service outsourcing, layered verification strategies, and the settlement flow.

1) Verification Triggering and Fee Management: This subsection defines only when verification is triggered, who pays, and how funds are settled. Verification can be triggered through two paths. The first is system spot-checking: the matching engine initiates random audits with a certain probability after task completion (Step 9.1), with spot-check fees prepaid by task requesters and deducted during settlement. The second is active challenge: if a task requester doubts the execution result, or a resource provider suspects anomalous behavior by its collaborators, either party may initiate a challenge by paying a challenge fee to trigger additional verification (Step 9.2). All fee transfers are managed by the arbitration contract Arbitration.sol: if a challenge is upheld, the challenge fee is refunded and compensation is paid from the misbehaving party's stake; if a challenge is rejected, the challenge fee is forfeited to the system pool. This design deters malicious challenges while preserving an appeal channel for honest participants.

2) Security Service Outsourcing via EigenLayer: Verification requires trusted execution entities and economic accountability. BC-RA adopts EigenLayer as the outsourcing infrastructure for this security service. EigenLayer is an Ethereum-based restaking protocol that allows validators to reuse staked ETH across multiple Actively Validated Services (AVS), thereby supplying slashing-backed eco-

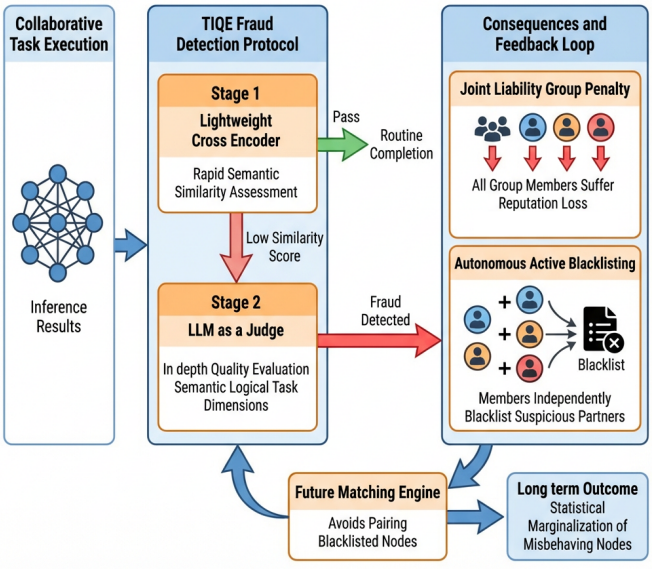

Fig. 3. TIQE fraud detection protocol with active blacklisting.

nomic guarantees for external services. In this framework, validators register as AVS operators and provide verification judgments; if operators misbehave, they are exposed to stake slashing. Hence, EigenLayer provides the economic security foundation of verification, while the concrete verification algorithms are defined by BC-RA itself.

3) Layered Verification Strategy: Under the security service framework provided by EigenLayer, BC-RA designs a layered progressive verification strategy to balance cost efficiency and precise accountability. Three verification layers are arranged in ascending order of cost and capability: a routine spot-check layer provides low-cost fraud screening with group penalties, a dispute-escalation layer precisely locates misbehaving nodes for arbitration, and a cryptographic assurance layer provides a long-term direction for stronger guarantees.

Routine Spot-Check Layer. As shown in Fig. 3, this layer serves as the default option for routine verification, comprising three components: fraud detection, group penalties, and active blacklisting.

For fraud detection, this work adopts the Two-stage Inference Quality Evaluation (TIQE) protocol inspired by PolyLink [12]. The first stage employs a lightweight cross-encoder model to perform rapid semantic similarity assessment of inference results, encoding the task input and inference output into vectors and computing a similarity score; if the score falls below a preset threshold, the process proceeds to the second stage. The second stage adopts an LLM-as-a-Judge mechanism, where an independent large language model performs in-depth quality evaluation of the inference result, comprehensively scoring across dimensions such as semantic consistency, logical coherence, and task completion. This two-stage design balances detection efficiency and accuracy: the first stage filters out most normal results to reduce overhead, while the second stage performs fine-grained judgment on suspicious results to minimize false positives.

It should be noted that the TIQE protocol can only determine whether a task as a whole involves fraud, but cannot precisely identify the specific misbehaving node. Therefore, this scheme adopts a design combining joint liability with active blacklisting.

For group penalties, when a task is determined to be fraudulent, all members of the collaborative group bear reputation loss. Let G denote the collaborative group of the fraudulent task; for all $i \in G$, the reputation score is updated as:

$$r_i \leftarrow r_i - \mu, \tag{14}$$

where $\mu > 0$ is the penalty coefficient. Since honest and misbehaving nodes cannot be distinguished, group penalties inevitably cause "collateral damage" to honest nodes.

For active blacklisting, collaborative group members may, based on their own judgment, add suspicious partners from the current collaboration to their blacklist. Let B_i denote node i's blacklist; if $j \in B_i$, the matching engine will avoid placing i and j in the same collaborative group for subsequent task assignments. Blacklisting decisions are made autonomously by nodes without system intervention.

This design induces spontaneous collaboration exclusion through game-theoretic mechanisms. After a task failure, honest nodes tend to blacklist suspicious partners who caused the failure. In the long run, truly misbehaving nodes are blacklisted by an increasing number of honest nodes due to frequently causing task failures, and their pool of potential collaborators shrinks progressively, ultimately making it difficult to form effective collaborative groups. Meanwhile, although honest nodes may occasionally suffer "collateral damage" and be blacklisted, their overall task success rate is higher, and their cumulative probability of being blacklisted is far lower than that of misbehaving nodes, ultimately achieving statistical marginalization of misbehaving nodes. Additionally, nodes with persistent misbehavior will have their fraud count f_i reach the ban threshold F_{ban}, resulting in direct task allocation refusal by the system. The advantage of this scheme is that the TIQE protocol does not require re-executing the complete inference task; fraud can be efficiently detected through semantic evaluation alone, with extremely low implementation cost. Its limitation is the inability to precisely locate misbehaving nodes, and the formation of exclusion effects requires accumulation over multiple rounds of gameplay.

Dispute-Escalation Layer. When the routine spot-check layer detects fraud and the system requires precise attribution for targeted penalties, BC-RA escalates to a chain-based verifier (Fig. 4). In implementation terms, this verifier is behind the escalation trigger interface (rather than always-on): after escalation, validators perform offline re-execution and submit the final verdict through the existing arbitration contracts.

Original THC baseline. We retain Tensor Hash Chain (THC) as the deterministic baseline. Let T_k denote the output tensor at shard boundary k, where $k \in \{1, 2, 3\}$ corresponds to checkpoints $C1, C2, C3$. THC is defined as:

$$h_1 = H(T_1), \quad h_k = H(h_{k-1} \| H(T_k)), \quad k \in \{2, 3\}, \quad (15)$$

where $H(\cdot)$ denotes a hash function and $\|$ denotes concatenation. During dispute verification, validators compare on-chain and re-executed chains checkpoint by checkpoint; the first mismatch provides shard-level suspicion localization. THC therefore offers strong chain-based accountability and precise first-mismatch attribution, but it assumes deterministic inference reproducibility across providers.

TSTC enhancement for heterogeneous execution. In heterogeneous hardware settings, honest executions may still show floating-point numerical divergence; direct full-tensor hashing can therefore over-trigger false alarms. To mitigate this issue, we introduce Tolerance-Aware Sampled Tensor Chain (TSTC). For shard-boundary checkpoint T_k, let Ω_k be a publicly reproducible sampling index set and define sampled values as

$$u_k = S_{\Omega_k}(T_k), \quad (16)$$

then apply tolerance-aware quantization

$$\tilde{u}_k = Q_{\Delta_k}(u_k), \quad (17)$$

and construct a block summary

$$s_k = H(\Omega_k \| \tilde{u}_k). \quad (18)$$

The chain commitment is

$$c_1 = H(s_1), \quad c_k = H(c_{k-1} \| s_k), \quad k \in \{2, 3\}. \quad (19)$$

In the final prototype narrative, dispute escalation is evaluated in a prefill-focused setting. Concretely, validators observe only prefill shard-boundary tensors $T_k \in \mathbf{R}^{B \times L \times H}$ at checkpoints $C1, C2, C3$, and TSTC uses token-channel stratified sampling on these tensors. The tolerance parameter Δ_k is checkpoint-specific and should be interpreted as a prototype-level calibrated slack under our heterogeneous stack rather than as a deployment-independent guarantee. This scope aligns the verifier with the provider-honesty scenario studied in the final experiments.

During dispute verification, validators recompute $\{c_k\}$ with the same Ω_k and Δ_k, then compare against on-chain commitments; the first mismatch index still supports shard-level localization. Relative to THC, TSTC compares tolerance-aware local stable features rather than requiring exact element-wise equality across devices. As a trade-off, TSTC reduces false alarms under honest heterogeneity but may miss attacks that avoid sampled coordinates; it is therefore a tolerance-aware verifier, not a strict semantic-correctness proof.

Cryptographic Assurance Layer. As a long-term evolution direction, zkLLM can generate cryptographic proofs of inference correctness without revealing model parameters or input data, while the DSperse framework reduces proof overhead through sharded verification [23], [24].

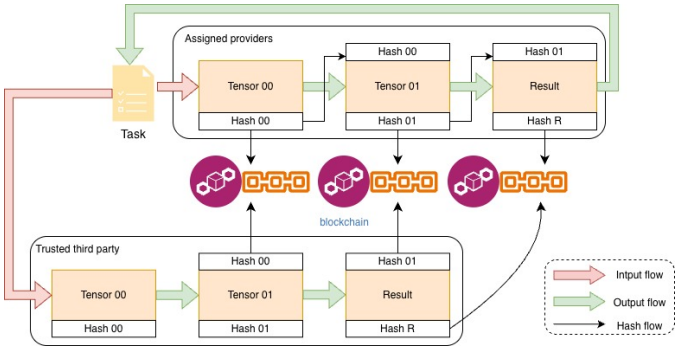

Fig. 4. Schematic diagram of chain-based dispute-escalation verification (THC baseline and TSTC enhancement).

This scheme can theoretically achieve the highest level of privacy protection and verification trustworthiness, requiring no trust in third parties to complete verification. However, under current technological conditions, proof generation overhead still far exceeds inference itself, making it difficult to apply directly to large-scale language models. This work discusses it as a future extension option.

In summary, the three layers constitute a progressive verification service menu: routine spot-checking uses TIQE screening with active blacklisting for cost efficiency; dispute escalation uses chain-based attribution (original THC baseline plus prefill-focused TSTC enhancement) for precise accountability under heterogeneous execution; and high-security scenarios can further consider a cryptographic verification path. This layered strategy balances day-to-day operating cost and attribution robustness in dispute scenarios.

4) Verification Results and Settlement Flow: After validators complete verification (via TIQE, THC baseline, or TSTC), they submit conclusions on-chain (Step 9.3). The arbitration contract then executes settlement according to four cases. First, no anomaly: the contract emits an "honest execution" signal, releases normal payment, and performs routine reputation updates (Step 10). Second, fraud/tamper confirmed: the contract records the misbehaving party (or first mismatched block owner for chain-based escalation), pays requester compensation, applies provider stake slashing, and increments fraud count f_i. Third, malicious challenge rejected: the challenger loses the challenge fee to the system pool. Fourth, verifier misconduct: the AVS operator enters the EigenLayer slashing path under restaking rules. All state transitions are executed atomically on-chain to keep economic consequences consistent with verification outcomes.

V. Experimental Results and Analysis

This section presents a comprehensive empirical evaluation of BC-RA along five dimensions: allocation and pricing quality, adversarial robustness, privacy overhead, end-to-end EXO deployment performance, and verification feasibility of dispute-escalation chain verifiers (original THC baseline and TSTC enhancement). We use a con-

sistent evaluation protocol and metric definition across all experiments to ensure fair comparison.

A. Allocation and Pricing Evaluation

We follow the notation in Table I and adopt the same mechanism setting as Section 4, namely the reputation-prioritized greedy matching engine (Algorithm 1) and the max-cost-based uniform pricing rule (Eq. (3)). Unless otherwise specified, $\alpha \in \{0.25, 0.5, 0.75\}$.

For clarity, we define the core allocation-pricing metrics used in this subsection. Let $q_j = \sum_{i \in S} x_{ij}$ denote realized allocation for task j. Requester and provider utilities are

$$U_j^B = (\theta_j - p_j) q_j, \qquad U_i^S = \sum_{j \in B} (p_j - a_i) x_{ij}. \tag{20}$$

The social welfare is computed as

$$W = \sum_{j \in B} U_j^B + \sum_{i \in S} U_i^S. \tag{21}$$

Define active requester/provider sets as $B^+ = \{j \in B \mid q_j > 0\}$ and $S^+ = \{i \in S \mid \sum_{j \in B} x_{ij} > 0\}$. Average utility metrics are

$$\bar{U}_B = \frac{1}{|B^+|} \sum_{j \in B^+} U_j^B, \qquad \bar{U}_S = \frac{1}{|S^+|} \sum_{i \in S^+} U_i^S. \tag{22}$$

1) Social Welfare: Figure 5 reports social welfare W under two scale regimes: buyer-side expansion (fixed $|S| = 50$) and provider-side expansion (fixed $|B| = 100$). Under buyer-side expansion, welfare increases with the buyer population and then gradually saturates in the high-demand region. Under provider-side expansion, welfare rises sharply as provider count grows from 30 to around 120, and then remains in a plateau around 3.3×10^4 to 3.5×10^4. Across both sweeps, varying α only induces limited separation compared with scale effects, indicating that aggregate welfare is primarily governed by feasibility and valuation–cost spread rather than surplus split.

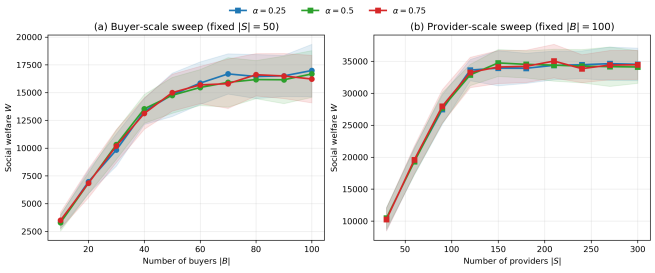

Fig. 5. Social welfare under buyer-side and provider-side scale sweeps, with ±1 standard-deviation bands.

2) Average Utility: Figure 6 (buyer-side scaling) and Figure 7 (provider-side scaling) show average utility of successful requesters and successful providers. Two stable patterns appear.

First, α consistently controls surplus distribution: larger α shifts price toward $a_j^{\max}$, increasing requester-side utility and reducing provider-side utility; smaller α does the opposite. Second, conditional utility changes with market regime: when buyers increase under fixed supply, successful requesters become more selective and their average utility rises, while successful providers face higher contention and lower per-capita gain; when providers increase under fixed demand, requester-side utility declines as lower-valuation requests enter the served set, and provider-side utility decreases due to supply-side competition.

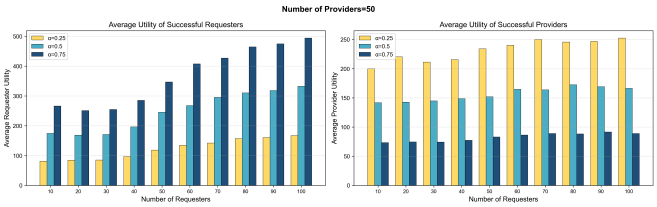

Fig. 6. Average utility under buyer-side expansion.

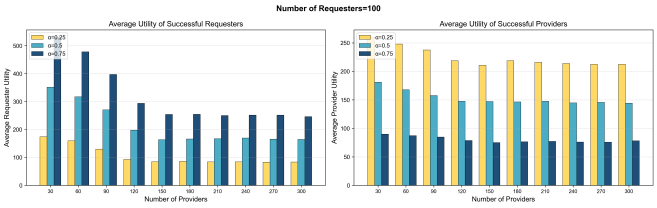

Fig. 7. Average utility under provider-side expansion.

3) Individual Rationality: We directly verify the individual rationality (IR) constraint of the uniform pricing mechanism. For any successfully matched requester j, the uniform unit price satisfies

$$\max_{i \in M_j} a_i \le p_j \le \theta_j. \tag{23}$$

As shown in Fig. 8, we visualize one representative run (with $\alpha = 0.5$) by sorting all 25 successfully matched requesters in descending θ_j and stacking three components: the maximum winner cost $a_{\max}$, the price premium $(p_j - a_{\max})$, and the requester surplus $(\theta_j - p_j)$. The figure reports zero IR violations, indicating that the pricing rule is not only theoretically constrained but also empirically stable under heterogeneous random instances, providing a minimal economic consistency guarantee for on-chain settlement and long-term operation.

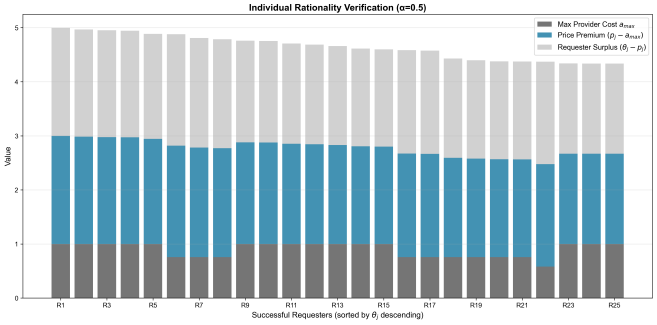

Fig. 8. Individual-rationality verification for matched tasks.

4) Computational Scalability: We evaluate the computational efficiency and scalability of the off-chain matching engine. The worst-case time complexity of the greedy matching procedure is $O(nm)$. We vary the total number of participants (providers + requesters) from 200 to 2000 and measure the running time of the matching and pricing stage only. Fig. 9 shows that the running time increases nonlinearly with system size, yet remains on the order of 1.4×10^{-1} seconds at 2000 participants. Combined with the complexity analysis, the results support that the mechanism is engineering-feasible with good throughput and scalability, making it suitable as a lightweight off-chain decision module that produces inputs for on-chain contracts.

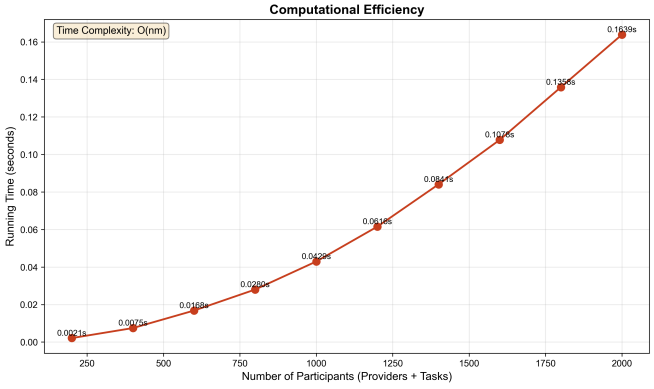

Fig. 9. Matching-pricing runtime versus total participants.

B. Adversarial Robustness

This subsection validates whether the mechanism-level robustness rules in Section 4 can reduce economic distortion under strategic misreports. The objective is empirical verification rather than introducing new mechanism patches at the experiment layer.

1) Attack Model and Experimental Setup: We use controlled Monte Carlo simulation with fixed matching/pricing logic and perturb only strategic reports. Unless otherwise stated, each market instance uses $|S| = 80$, $|B| = 120$, and $\alpha = 0.5$. For each setting, we run 30 independent rounds and report mean ± std.

We evaluate two adversarial models.

- Provider markup attack. An adversarial subset $A_S \subseteq S$, $|A_S| = \rho|S|$, inflates asks:

$$a'_i = \lambda a_i, \quad i \in A_S, \tag{24}$$

with $\rho \in \{0.1, 0.2, 0.3\}$, $\lambda \in \{1.0, 1.2, 1.4, 1.6, 1.8\}$. We compare no-guard pricing and the guarded pricing rule in Eq. (7).

- Requester demand inflation attack. An adversarial subset $A_B \subseteq B$, $|A_B| = \rho|B|$, over-reports demand:

$$d'_j = \gamma d_j, \quad j \in A_B, \tag{25}$$

with $\rho \in \{0.1, 0.2, 0.3\}$, $\gamma \in \{1.0, 1.25, 1.5, 1.75, 2.0\}$. We compare no guard and the demand sanity cap in Eq. (10).

Let superscripts (0) and (1) denote outcomes before and after attack injection on the same random instance. For markup attack:

$$L_{\text{req}} = U_B^{(0)} - U_B^{(1)}, \tag{26}$$

$$I_p = \frac{\bar{p}^{(1)} - \bar{p}^{(0)}}{\bar{p}^{(0)}}, \tag{27}$$

where $U_B = \sum_{j \in B} U_j^B$, $\bar{p} = \frac{1}{|B^+|} \sum_{j \in B^+} p_j$. For demand inflation, with attacked allocation $\hat{q}_j = \sum_i x_{ij}^{\text{attack}}$:

$$R_{\text{mis}} = \frac{\sum_{j \in B} \max(0, \hat{q}_j - d_j)}{\sum_{j \in B} \hat{q}_j}, \tag{28}$$

$$R_{\text{sat}} = \frac{\sum_{j \in B} \min(\hat{q}_j, d_j)}{\sum_{j \in B} d_j}, \tag{29}$$

and welfare degradation:

$$\Delta W = W^{\text{clean}} - W^{\text{attack,true}}, \tag{30}$$

$$W^{\text{attack,true}} = \sum_{j \in B} \theta_j \min(\hat{q}_j, d_j) - \sum_{i \in S} \sum_{j \in B} a_i x_{ij}^{\text{attack}}. \tag{31}$$

2) Results: Figure 10 reports full sweeps. Tables II and III summarize the strongest tested settings.

For provider markup attack, both L_{req} and I_p increase with λ, and the slope becomes steeper at larger ρ. At $(\rho = 0.3, \lambda = 1.8)$, guarded pricing reduces requester loss from 950.98 ± 275.06 to 853.77 ± 275.61 (-10.2%) and price inflation from $5.01\% \pm 1.28\%$ to $4.47\% \pm 1.22\%$ (-10.9%). Social welfare remains unchanged because this guard modifies only the price reference, not allocation.

For requester demand inflation, R_{mis} and ΔW increase with γ, while R_{sat} decreases. At $(\rho = 0.3, \gamma = 2.0)$, demand sanity cap reduces misallocation from $23.3\% \pm 3.7\%$ to $21.1\% \pm 3.9\%$ (-9.4%), lowers welfare drop from 6730.5 ± 1277.6 to 6102.2 ± 1385.1 (-9.3%), and increases true-demand satisfaction from $41.2\% \pm 5.0\%$ to $42.3\% \pm 4.9\%$ $(+2.8\%)$.

3) Discussion: Two observations are operationally important. First, both guards show targeted robustness: they reduce attack-induced distortion while preserving clean-setting behavior (for demand inflation, guard and no-guard curves overlap at low γ, then diverge only when inflation exceeds normal demand tails). Second, robustness is bounded but not absolute: strategic misreports still degrade outcomes at high attack levels, indicating that robust clipping improves resilience but does not make the mechanism strategy-proof.

TABLE II
Provider markup attack at strongest setting $(\rho = 0.3, \lambda = 1.8)$, reported as mean ± std over 30 rounds.

Pricing mode	L_{req}	I_p	W	Guard activation
Baseline	950.98 ± 275.06	$5.01\% \pm 1.28\%$	24720.63 ± 2161.29	N/A
Guarded pricing (Eq. (7))	853.77 ± 275.61	$4.47\% \pm 1.22\%$	24720.63 ± 2161.29	11/30 (36.7%)
Relative change	-10.2%	-10.9%	0.0%	—

TABLE III
Requester demand inflation at strongest setting ($\rho = 0.3, \gamma = 2.0$), mean $\pm$ std over 30 rounds.

Demand mode	R_{mis}	ΔW	R_{sat}	Guard activation
No guard	$23.3\% \pm 3.7\%$	6730.5 ± 1277.6	$41.2\% \pm 5.0\%$	N/A
Demand sanity cap (Eq. (10))	$21.1\% \pm 3.9\%$	6102.2 ± 1385.1	$42.3\% \pm 4.9\%$	30/30
Relative change	-9.4%	-9.3%	$+2.8\%$	Avg capped buyers = 5.2

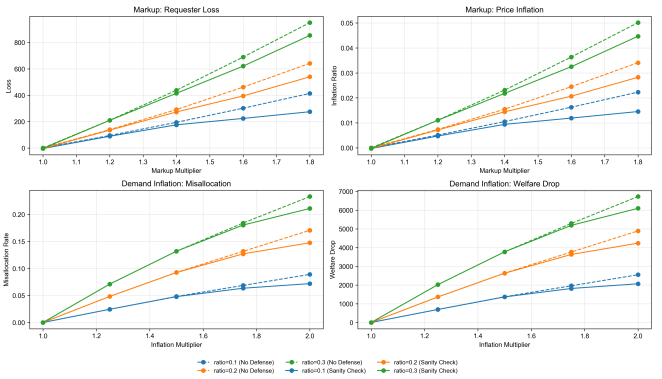

Fig. 10. Adversarial robustness under provider markup and requester demand inflation attacks, with and without mechanism-level guards from Section 4.

C. Privacy Mechanism Evaluation

1) End-to-End Latency: We compare two protocols under the same network model and transmission abstraction. The proposed protocol is Publish-Once, Key-to-Winners (POKW), which publishes one ciphertext to decentralized storage, records the plaintext commitment $H(m_j)$ on-chain, and distributes encrypted (ℓ_j, k_j) only to winners. The baseline is Replicated Payload Delivery (RPD), which sends a full encrypted payload independently to each winner.

The experiment uses payload sizes $\{1, 5, 10, 50, 100\}$ MB, fixed winner-set size $|M_j| = 3$, and $N = 5$ repeated runs per configuration. The LAN profile is configured as $(1000\,\text{Mbps}, 1\,\text{ms}, 0)$, and the WAN profile as $(100\,\text{Mbps}, 50\,\text{ms}, 0.01)$, corresponding to bandwidth, RTT, and packet-loss rate.

For task j, end-to-end latency is defined as

$$T_{\text{e2e}} = T_{\text{req}} + \max_{i \in M_j} T_{\text{prov},i}. \tag{32}$$

Requester-side time is decomposed as

$$T_{\text{req}}^{\text{POKW}} = T_1 + T_2 + T_3 + T_4 + T_5, \tag{33}$$
$$T_{\text{req}}^{\text{RPD}} = T_1' + T_2' + T_3' + T_4'. \tag{34}$$

For each network and payload configuration, we report median and interquartile range. The relative latency gain of POKW over RPD is

$$\text{Gain}_{\text{lat}} = \frac{\tilde{T}_{\text{e2e}}^{\text{RPD}} - \tilde{T}_{\text{e2e}}^{\text{POKW}}}{\tilde{T}_{\text{e2e}}^{\text{RPD}}}, \tag{35}$$

where $\tilde{T}_{\text{e2e}}$ denotes the median latency.

As shown in Figure 11, POKW achieves consistently lower latency at larger payload sizes. At 100 MB, the median latency decreases from 2.668 s to 1.828 s in LAN and from 25.549 s to 17.317 s in WAN, corresponding to 31.5% and 32.2% gains.

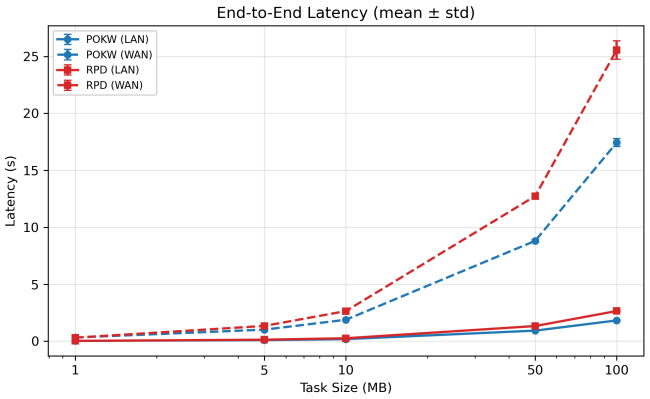

Fig. 11. End-to-end latency under POKW and RPD across payload sizes in LAN and WAN.

2) Time Breakdown: To explain the latency gap, we decompose requester-side time into the components listed in Table IV. For each component k, we report its relative share

$$r_k = \frac{T_k}{T_{\text{req}}}, \tag{36}$$

which identifies dominant requester-side bottlenecks.

TABLE IV
Requester-side time components for POKW and RPD.

Protocol	Symbol	Meaning
POKW	T_1	symmetric key generation
	T_2	symmetric encryption
	T_3	ciphertext upload to IPFS
	T_4	plaintext commitment hash generation $H(m_j)$
	T_5	targeted encrypted key delivery
RPD	T_1'	key generation
	T_2'	symmetric encryption
	T_3'	per-winner key encapsulation
	T_4'	replicated ciphertext transmission

Figure 12 shows that POKW is dominated by one-time ciphertext publication, whereas RPD is dominated by replicated ciphertext transmission. For RPD, the dominant transfer term T_4' scales as $\mathcal{O}(|M_j| \cdot |m_j|)$. For POKW, the dominant publication term T_3 scales as $\mathcal{O}(|m_j|)$, and targeted key delivery operates on a much smaller payload than ciphertext transfer. This difference explains the widening gap under larger payload sizes.

D. EXO Implementation Evaluation

This subsection evaluates the deployment feasibility of BC-RA with EXO after requester–provider matching has already been completed. We focus on two questions: 1) whether the end-to-end user experience remains acceptable under both LAN and WAN conditions, and 2) whether increasing the number of EXO instances improves parallel serving capability when provider resources are sufficient.

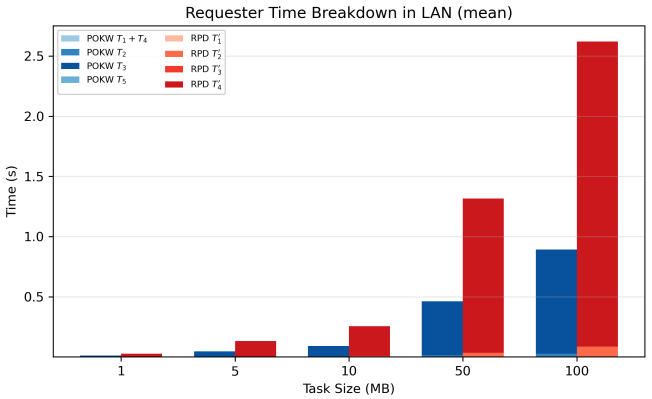

Fig. 12. Requester-side time breakdown in LAN for POKW and RPD.

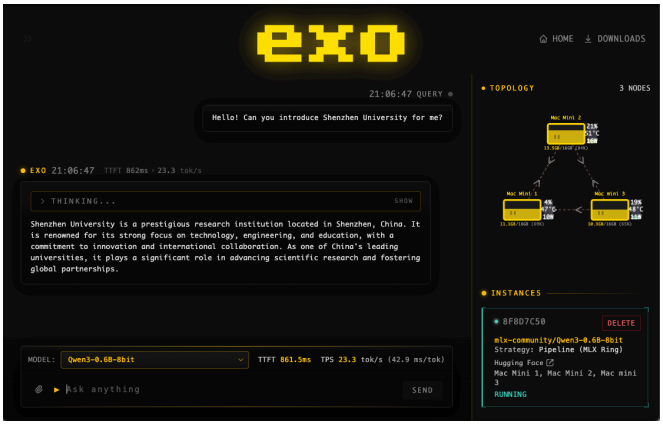

Fig. 13. EXO instance abstraction and deployment used in the feasibility evaluation.

The deployment uses one MacBook requester and three Mac mini providers (Fig. 13). All providers run the same Qwen3 0.6B model. For each experimental cell, the requester issues one task containing 100 inference prompts, while the task data are published through IPFS and fetched by the selected providers before inference. We vary the active EXO instance count $n_{\text{inst}} \in \{1, 2\}$ and the network condition $\{\text{LAN}, \text{WAN}\}$. Here, an EXO instance denotes a runnable model deployment unit rather than thread-level concurrency inside one serving process [7]; increasing n_{inst} therefore adds parallel service units instead of merely injecting more requester-side requests.

The LAN setting places all four devices in the same Wi-Fi network. The WAN setting is emulated on macOS by applying per-provider RTT/bandwidth/loss constraints through pf+dummynet, which provides controlled network emulation over the real protocol stack and is widely used for protocol and system benchmarking [29], [30]. We use three heterogeneous emulated links to represent the requester-to-provider WAN paths:

- Link 1: RTT 22 ms, bandwidth 120 Mbps, loss 0.5%.
- Link 2: RTT 45 ms, bandwidth 60 Mbps, loss 1.5%.
- Link 3: RTT 80 ms, bandwidth 40 Mbps, loss 2.0%.

We report the following metrics:

- Task Latency (s): total elapsed time from a provider starting task-data download from IPFS to the requester receiving the complete response.
- Download (s): time for the provider to finish downloading the task data from IPFS.
- TTFT (s): elapsed time from request launch to the first returned token.
- OTPS (tok/s): output-token throughput during the decoding stage.
- RTPS (req/s): completed-request throughput over the measurement window.

For each metric m, we compare both WAN-versus-LAN and two-instance-versus-one-instance settings using the relative difference $\Delta m/m_{\text{base}}$.

Table V is the reporting template for the revised study. The design is intended to test two hypotheses rather than claim measured gains at this stage. First, WAN is expected to degrade the latency-oriented metrics (Task Latency, Download, and TTFT), while still remaining potentially acceptable for personal-scale usage if the absolute delay stays within an interactive tolerance band. Second, increasing the instance count is expected to improve RTPS and overall parallel resource utilization, but it may also increase Task Latency and TTFT because of extra scheduling overhead and inter-instance resource contention. These hypotheses require confirmation from the completed measurements.

TABLE V
Reporting template for EXO deployment feasibility evaluation under different instance counts and network conditions.

Instance Count	Network	Task Latency (s)	Download (s)	TTFT (s)	OTPS (tok/s)	RTPS (req/s)
1	LAN	–	–	–	–	–
1	WAN	–	–	–	–	–
2	LAN	–	–	–	–	–
2	WAN	–	–	–	–	–

E. Verification Feasibility Evaluation

Following the layered verification design in Section 4, this subsection evaluates only the dispute-escalation verifiers; TIQE remains the routine screening layer and is not benchmarked here. We adopt a prefill-focused verification setting and ask a narrow question: can a verifier confirm that providers honestly execute the assigned shard plan during prefill while remaining tolerant to heterogeneous but honest execution?

The experiment uses the Qwen3 0.6B model under a fixed three-provider shard plan and takes Shard k as the verification unit. The study is conducted on a prototype heterogeneous execution harness spanning two Mac mini M4 devices and one Linux RTX3090 server. The verifier observes only three prefill shard-boundary checkpoints:

- C1: output of Shard 1 before transfer to Shard 2;
- C2: output of Shard 2 before transfer to Shard 3;
- C3: final hidden output of Shard 3.

Each checkpoint tensor therefore has shape $B \times L \times H$ on the prefill path.

We compare two dispute-escalation verifiers:

- THC: direct FP32 tensor hashing using Eq. (15).
- TSTC: tolerance-aware sampled chain using Eqs. (16)–(19).

We evaluate three scenarios: Honest-Homo, Honest-Hetero, and Tamper. We use disjoint calibration and evaluation prompt splits: tolerance values are fixed before reporting, and all final metrics are computed on the evaluation split only. For verifier v, let $\mathcal{R}_v^{\text{tamper}}$ and $\mathcal{R}_v^{\text{hetero}}$ denote the tamper and honest-hetero trial sets, respectively. We report

$$\text{TPR}_v = \frac{1}{|\mathcal{R}_v^{\text{tamper}}|} \sum_{r \in \mathcal{R}_v^{\text{tamper}}} \mathbf{1}[\text{detected}_r], \tag{37}$$

$$\text{FPR}_v = \frac{1}{|\mathcal{R}_v^{\text{hetero}}|} \sum_{r \in \mathcal{R}_v^{\text{hetero}}} \mathbf{1}[\text{detected}_r], \tag{38}$$

$$\text{LocAcc}_v = \frac{1}{|\mathcal{R}_v^{\text{tamper}}|} \sum_{r \in \mathcal{R}_v^{\text{tamper}}} \mathbf{1}[\hat{k}_r = k_r^\star], \tag{39}$$

where $k_r^\star$ is the true first tampered shard boundary and $\hat{k}_r$ is the verifier-reported first mismatch checkpoint. Honest-Hetero false positives are additionally broken down by heterogeneity level (low, mid, high).

The final evaluation considers prefill checkpoints only and uses a fixed small stratified sample of token positions and hidden channels for TSTC, corresponding to 16 sampled activations per checkpoint. It contains 300 evaluation prompts and 10 repeated trials per mode. The tuned prefill tolerance map is $\Delta_{C1} = 0.0022$, $\Delta_{C2} = 0.00525$, and $\Delta_{C3} = 0.02$. The heterogeneous execution profile combines 8-bit Metal on the first shard, BF16 on the second shard, and FP32 on the final shard.

Fig. 14 summarizes the main comparison. On Honest-Homo, both THC and TSTC remain clean with FPR = 0. On Tamper, both verifiers achieve TPR = 1.0 and LocAcc = 1.0, indicating that tolerance does not weaken detection or first-mismatch localization in this prototype. The substantive separation appears on Honest-Hetero: THC remains at FPR = 1.0, whereas TSTC reduces the false-positive rate to 0.170111. Fig. 15 shows that this advantage is stable across all three heterogeneity levels, with TSTC yielding 0.151333 for low, 0.175333 for mid, and 0.183667 for high.

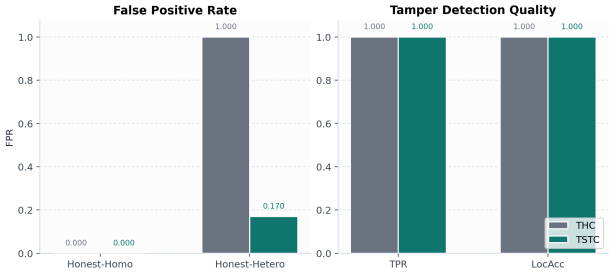

Fig. 14. Prefill-focused verifier comparison on the evaluation split. Left: false-positive rates under Honest-Homo and Honest-Hetero. Right: tamper detection rate and localization accuracy.

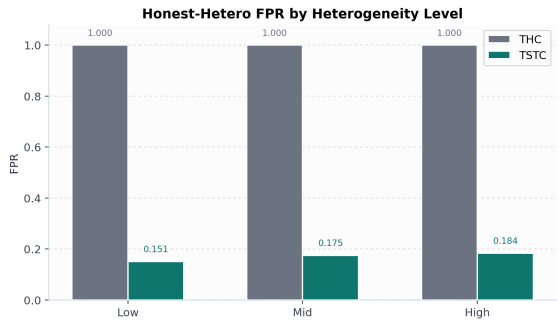

Fig. 15. Prefill-focused Honest-Hetero false-positive rate across heterogeneity levels.

Taken together, these results support a clear claim. In the current evaluation object, a prefill-focused TSTC substantially reduces false alarms relative to strict THC while preserving shard-level tamper detection and localization. The evidence remains prototype-level rather than deployment-level, however, because the Honest-Hetero condition is constructed by synthetic perturbation on captured bundles rather than by paired real-capture honest-versus-honest comparison.

VI. Discussion and Limitations

This section consolidates the key design trade-offs and limitations of BC-RA, providing transparency about the boundaries of our contributions.

The max-cost-based uniform pricing mechanism does not guarantee strategy-proofness. Providers may strategically inflate their ask prices to increase the uniform price, potentially capturing additional surplus at the expense of requesters. While reputation-based ranking and long-term stake exposure provide partial mitigation, these mechanisms do not eliminate strategic behavior entirely. Under balanced or over-supplied market conditions, providers with inflated asks are less likely to be selected, and those who consistently fail to win allocations accumulate fewer completed tasks, resulting in lower reputation scores. Nonetheless, formal incentive compatibility remains an open challenge, and future work may explore VCG-based mechanisms or double auctions with strategy-proof properties.

The TIQE-based fraud detection layer can only determine whether a task as a whole involves fraud, without identifying the specific misbehaving node. Consequently, group penalties inevitably cause collateral damage to honest nodes who happen to collaborate with a misbehaving partner. However, over multiple rounds, honest nodes accumulate higher task success rates and lower cumulative penalties, while persistent misbehavers are progressively excluded through active blacklisting. This trade-off between detection efficiency and precise attribution is intentional: TIQE avoids the cost of full re-execution required by chain-based escalation verifiers, making it suitable for routine spot-checking at scale. The present empirical section focuses on dispute-escalation verifier

comparison (THC baseline and TSTC enhancement); a fully standardized TIQE benchmark remains future work.

The original Tensor Hash Chain (THC) baseline assumes deterministic inference execution, which may not hold in heterogeneous edge environments. Tensor hashes are reproducible only if all providers use fixed numerical precision, deterministic kernels, and standardized sharding schemes. Different hardware architectures or kernel stacks may produce slightly different floating-point results due to non-associative arithmetic, potentially causing hash mismatches even for honest execution. In the final prototype narrative, we therefore narrow the dispute-escalation problem to prefill-focused verification: validators compare only three shard-boundary prefill checkpoints and allow checkpoint-specific tolerance through TSTC. Under this scope, the tuned prototype reduces Honest-Hetero false positives from 1.0 with THC to 0.170111 with TSTC while preserving TPR = 1.0 and LocAcc = 1.0 on tamper. The remaining limitation is interpretive rather than purely numerical: the current Honest-Hetero condition is still generated by synthetic perturbation on captured bundles, so the result should be read as prototype-level evidence for prefill verification rather than as a full proof of heterogeneous deployment robustness.

BC-RA relies on stake-based identity binding to deter Sybil attacks, where a single entity creates multiple identities to manipulate the system. While staking raises the cost of mounting such attacks, it does not provide cryptographic Sybil resistance. An attacker could create multiple identities to execute low-value tasks, building reputation before launching coordinated misbehavior, or multiple Sybil identities may collude to form collaborative groups that escape TIQE detection. Robust Sybil resistance mechanisms such as decentralized identity protocols or proof-of-personhood remain important directions for future work.

Our experimental evaluation focuses on simulation-based analysis of matching, pricing, and privacy mechanisms, while the revised verifier study now provides a concrete artifact pipeline for prefill checkpoint capture, tolerance tuning, and evaluation. The verifier experiments are supported by a prototype heterogeneous execution harness and therefore should be interpreted as mechanism-level validation rather than as proof of complete exo-native deployment. Gas costs and storage overhead are not empirically measured but based on complexity estimates. Task and provider characteristics are generated from parametric distributions rather than real-world traces, and long-term reputation dynamics across multiple rounds are not fully captured. Future work should include paired real-capture honest-versus-honest verification, broader multi-device calibration, more diverse kernel/backend combinations, exo-native verifier integration, and longitudinal studies of participant behavior.

VII. Conclusion

This paper presents BC-RA, a blockchain-based resource allocation mechanism for edge AI networks that addresses trust, incentive, and privacy challenges in decentralized computing environments. Rather than pursuing strict mechanism-theoretic optimality, BC-RA prioritizes deployability, individual rationality, and system composability. The system adopts a tri-layer architecture integrating on-chain contracts, off-chain matching engines, and a verification layer.

The core contributions include three aspects. First, we design a reputation-aware greedy matching algorithm with max-cost-based uniform pricing, balancing allocation efficiency with long-term fairness while guaranteeing individual rationality for both sides. Second, we propose a privacy-preserving task distribution strategy combining off-chain storage, on-chain commitment, and targeted key distribution, which reduces end-to-end latency by approximately 30% and upload bandwidth to $1/|M_j|$ for large-scale tasks. Third, we develop a layered verification framework leveraging EigenLayer, comprising TIQE-based fraud detection, chain-based dispute escalation using a strict THC baseline and a prefill-focused TSTC enhancement, and zero-knowledge verification as a long-term future direction. The revised verifier study completes a real three-machine prototype pipeline for checkpoint capture, tolerance tuning, and evaluation. In the accepted prefill-focused setting, TSTC reduces Honest-Hetero false positives from 1.0 to 0.170111 while preserving perfect tamper detection and shard-level localization in our prototype.

We acknowledge several limitations discussed in Section 6, including the lack of strategy-proofness in pricing, collateral damage from group penalties, deterministic-profile assumptions in the original THC baseline, sampling-induced trade-offs in TSTC, and the need for stronger Sybil resistance. These trade-offs reflect our intentional prioritization of engineering feasibility over theoretical optimality.

Future work includes exploring auction-based matching approaches with stronger incentive guarantees, extending TSTC evaluation to paired real-capture honest-versus-honest settings, developing robust Sybil resistance mechanisms, extending the system to support cross-chain interoperability, and integrating the verifier path into an exo-native runtime. In particular, the next verifier milestone is to test whether the prefill-focused tolerance learned in the current prototype transfers to broader heterogeneous hardware profiles without sacrificing localization fidelity.

References

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.

[2] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, 2021, pp. 610–623.

[3] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arber, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill et al., "On the opportunities and risks of foundation models," arXiv preprint arXiv:2108.07258, 2021.

[4] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, and A. Y. Zomaya, "Edge intelligence: The confluence of edge computing and artificial intelligence," IEEE Internet of Things Journal, vol. 7, no. 8, pp. 7457–7469, 2020.

[5] T. Meuser, L. Lovén, M. Bhuyan, S. G. Patil, S. Dustdar, A. Aral, S. Bayhan, C. Becker, E. De Lara, A. Y. Ding et al., "Revisiting edge ai: Opportunities and challenges," IEEE Internet Computing, vol. 28, no. 4, pp. 49–59, 2024.

[6] Z. Wang, R. Sun, E. Lui, V. Shah, X. Xiong, J. Sun, D. Crapis, and W. Knottenbelt, "Sok: Decentralized ai (deai)," arXiv preprint arXiv:2411.17461, 2024.

[7] A. Cheema, "Exo github," https://github.com/exo-explore/exo, 2025, accessed: 2025-06-18.

[8] ——, "Exo blog," https://blog.exolabs.net/day-1, 2025, accessed: 2025-06-18.

[9] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008, accessed: 2025-11-22.

[10] E. Bellini, Y. Iraqi, and E. Damiani, "Blockchain-based distributed trust and reputation management systems: A survey," IEEE Access, vol. 8, pp. 21 127–21 151, 2020.

[11] G. Baranwal, D. Kumar, and D. P. Vidyarthi, "Blockchain based resource allocation in cloud and distributed edge computing: A survey," Computer Communications, vol. 209, pp. 469–498, 2024.

[12] H. Liu, J. Cao, B. Yang, D. Bai, Y. Cao, X. Shen, Y. Zhang, J. Liang, S. Jiang, and M. Zhang, "Polylink: A blockchain based decentralized edge ai platform for llm inference," arXiv preprint, 2025, available at https://github.com/IMCL-PolyLink/PolyLink.

[13] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[14] J. Benet, "Ipfs - content addressed, versioned, p2p file system," arXiv preprint arXiv:1407.3561, 2014.

[15] E. Team, "Eigenlayer: Restaking protocol," https://docs.eigenlayer.xyz/, 2024, accessed: 2025-01-XX.

[16] H. Sun, J. Li, and H. Zhang, "zkllm: Zero knowledge proofs for large language models," in Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 4405–4419.

[17] D. Ivanov, T. Freiberg, S. Shahabi, J. Gold, and H. Isah, "Dsperse: A framework for targeted verification in zero-knowledge machine learning," arXiv preprint arXiv:2508.06972, 2025.

[18] W. Li, J. Wu, L. Zhang, K. Wang, and N. Cheng, "Double auction mechanisms in edge computing resource allocation for blockchain networks," Cluster Computing, vol. 27, no. 4, pp. 3017–3035, 2024.

[19] U. Habiba, S. Maghsudi, and E. Hossain, "A repeated auction model for load-aware dynamic resource allocation in multi-access edge computing," IEEE Transactions on Mobile Computing, vol. 23, no. 9, pp. 7801–7817, 2024.

[20] L. Javet, N. Anciaux, L. Bouganim, and P. Pucheral, "Edgelet computing: Enabling privacy-preserving decentralized data processing at the network edge," Personal and Ubiquitous Computing, vol. 29, pp. 45–75, 2025.

[21] Q. Li, C. Wang, X. Jiang, and J. Ma, "Approximate homomorphic encryption based privacy-preserving machine learning: a survey," Artificial Intelligence Review, vol. 58, no. 1, p. 82, 2025.

[22] M. U. Sardar, M. K. Khan, M. Alazab, and A. Y. Zomaya, "Confidential computing: a security and privacy-preserving paradigm for trusted execution environments," IEEE Transactions on Information Forensics and Security, vol. 18, pp. 5125–5142, 2023.

[23] Z. Xing, Z. Zhang, Z. Zhang, Z. Li, M. Li, J. Liu, Z. Zhang, Y. Zhao, Q. Sun, L. Zhu et al., "Zero-knowledge proof-based verifiable decentralized machine learning in communication network: A comprehensive survey," IEEE Communications Surveys & Tutorials, 2025.

[24] Z. Peng, T. Wang, C. Zhao, G. Liao, Z. Lin, Y. Liu, B. Cao, L. Shi, Q. Yang, and S. Zhang, "A survey of zero-knowledge proof based verifiable machine learning," arXiv preprint arXiv:2502.18535, 2025.

[25] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum project yellow paper, vol. 151, pp. 1–32, 2014.

[26] V. Buterin, "A next-generation smart contract and decentralized application platform," Ethereum White Paper, 2014.

[27] R. Biswas, X. Lu, and D. K. Panda, "Designing a micro-benchmark suite to evaluate grpc for tensorflow: Early experiences," CoRR, vol. abs/1804.01138, 2018. [Online]. Available: http://arxiv.org/abs/1804.01138

[28] D. Trautwein, C. Ihle, M. Schubotz, and B. Gipp, "Challenging tribal knowledge–large scale measurement campaign on decentralized nat traversal," arXiv preprint arXiv:2510.27500, 2025.

[29] L. Rizzo, "Dummynet: A simple approach to the evaluation of network protocols," ACM SIGCOMM Computer Communication Review, vol. 27, no. 1, pp. 31–41, 1997.

[30] S. Bradner and J. McQuaid, "Benchmarking methodology for network interconnect devices," RFC 2544, IETF, 1999, available: https://www.rfc-editor.org/rfc/rfc2544.